

PROJECT REPORT

EXPERIMENT - 5

Project Statement

Design and develop a Water Level Detection and Flood Alert System using Ultrasonic Sensor. The system measures water levels in rivers, tanks, or dams and provides early warning alerts during overflow conditions.

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

Water Level Detection and Flood Alert System Using Ultrasonic Sensor

Project Overview

The Water Level Detection and Flood Alert System is an IoT-based safety system designed to monitor the water level in rivers, tanks, dams, and other water bodies. An HC-SR04 ultrasonic sensor measures the distance between the sensor and the water surface. The ESP8266 Node MCU processes the sensor data and determines whether the water level is normal, high, or critical.

When the water reaches a predefined danger level, the system activates LED indicators and a buzzer to provide a local warning. A GSM module can also be used to send an alert message to a predefined mobile number, enabling early warning and faster response during possible overflow or flood conditions.

Main Objectives:

- To continuously monitor the water level.
- To detect abnormal or rapidly increasing water levels.
- To provide an early warning before overflow occurs.
- To activate visual and audio alerts during critical conditions.
- To send an alert through the GSM module.
- To develop a low-cost and simple flood-monitoring prototype.
- To improve safety around tanks, dams, and water bodies.

Problem Statement

Traditional water-level monitoring systems may require manual observation, which can result in delayed detection of dangerous water levels. During heavy rainfall or sudden water inflow, the water level can rise rapidly and cause overflow or flooding.

Therefore, a system is required that can automatically measure the water level, identify critical conditions, and provide immediate alerts. The proposed system uses an ultrasonic sensor, ESP8266 Node MCU, buzzer, LEDs, and GSM module to provide automatic water-level monitoring and early flood warnings.

Problems Addressed:

- Manual water-level monitoring.
- Delayed flood detection.
- Lack of continuous monitoring.
- Absence of immediate local warning.
- Difficulty in monitoring remote water bodies.
- Risk of overflow in tanks and dams.
- Delayed communication during critical conditions.

Proposed Solution

The proposed system places an HC-SR04 ultrasonic sensor above the water surface. The sensor sends ultrasonic waves toward the water and measures the time taken for the echo to return.

The ESP8266 calculates the distance to the water surface and determines the corresponding water-level condition. Two LEDs indicate the water condition, while the buzzer provides an audible warning. When the water level reaches the critical threshold, the GSM module sends an alert message to the registered user.

Proposed Operation:

The system operates as follows:

Ultrasonic Sensor → ESP8266 → Water-Level Calculation → Level Comparison → LED/Buzzer Alert → GSM Alert

The water level is continuously measured. If the measured level is within the safe range, the system operates normally. When the water level rises above the warning threshold, the warning LED is activated. When it reaches the critical threshold, the buzzer and critical LED are activated and an alert message is sent through GSM.

System Architecture

Input Section:

- HC-SR04 Ultrasonic Sensor detects the distance between the sensor and the water surface.
- The measured distance represents the current water-level condition.
- The sensor continuously provides input data to the ESP8266 NodeMCU.

Processing Section:

- ESP8266 Node MCU acts as the main controller.
- It receives the distance data from the ultrasonic sensor.
- It calculates the water level and compares it with predefined threshold values.
- It determines whether the condition is Normal, Warning, or Critical.

Actuation Section:

- Buzzer acts as the main warning actuator.
- When the water level reaches the critical limit, the ESP8266 activates the buzzer.
- The GSM module sends an alert message to the registered user during a critical condition.

Indication Section:

- **LED 1** – Normal Indicator: Indicates that the water level is within the safe range.
- **LED 2** – Warning/Critical Indicator: Indicates that the water level has reached a dangerous range.
- The buzzer provides an additional audible indication during critical conditions.

Mechanical Section:

- A water tank/container is used as the prototype water body.
- The HC-SR04 sensor is mounted securely above the water surface.

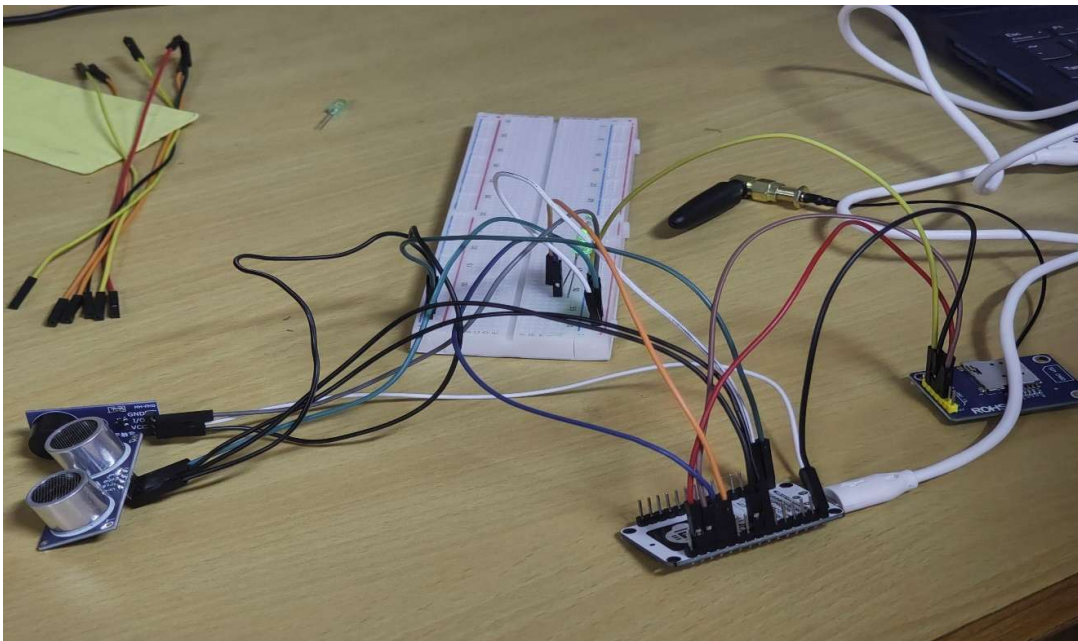
- The sensor position is maintained to obtain consistent distance measurements.
- The breadboard holds the ESP8266, LEDs, buzzer, and wiring for the prototype setup.

Overall Flow:

Mechanical Section → Input Section → Processing Section → Actuation Section
→ Indication Section

Water Surface → HC-SR04 → ESP8266 → Buzzer/GSM → LEDs & Alert Message

Image:



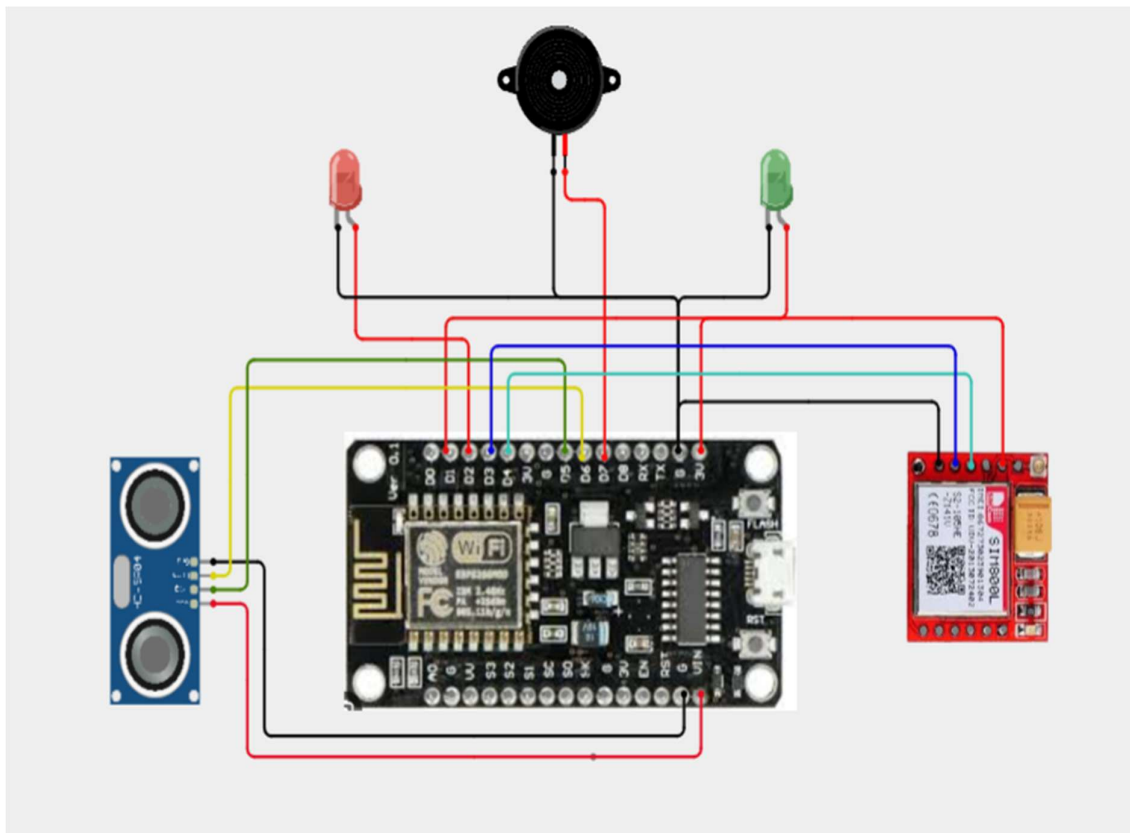
Circuit table

S. No.	Component	Pin/Terminal	ESP8266 Node MCU Connection
1	HC-SR04	VCC	5V/VU
		GND	GND
		TRIG	D5 (GPIO14)
		ECHO	D6 (GPIO12) through voltage divider
2	Normal LED	Anode (+)	D1 (GPIO5) through resistor
		Cathode (−)	GND
3	Warning LED	Anode (+)	D2 (GPIO4) through resistor
		Cathode (−)	GND
4	Buzzer	Positive (+)	D7 (GPIO13)
		Negative (−)	GND
5	GSM Module	TX	ESP8266 RX
		RX	ESP8266 TX
		GND	GND
		VCC	Suitable external supply
6	ESP8266 Node MCU	GND	Common GND

Important Pin Mapping:

- **HC-SR04 TRIG** → **D5 (GPIO14)** – Sends the ultrasonic trigger signal.
- **HC-SR04 ECHO** → **D6 (GPIO12)** – Receives the reflected ultrasonic signal through a voltage divider.
- **Normal LED** → **D1 (GPIO5)** – Indicates the safe water level.
- **Warning LED** → **D2 (GPIO4)** – Indicates warning/critical water level.
- **Buzzer** → **D7 (GPIO13)** – Provides an audible alert during critical conditions.
- **GSM TX** → **ESP8266 RX** – Transfers data from GSM to Node MCU.
- **GSM RX** → **ESP8266 TX** – Transfers data from Node MCU to GSM.
- **All GND pins** → **GND** – Provides a common ground for the circuit.

Circuit Design



Algorithm

- Start the system.
- Initialize ESP8266, ultrasonic sensor, LEDs, buzzer, and GSM module.
- Trigger the HC-SR04 sensor.
- Measure the echo return time.
- Calculate the distance between the sensor and water surface.
- Convert the measured distance into water-level status.
- Compare the water level with predefined thresholds.
- If the level is safe, activate the normal indication.
- If the level reaches the warning region, activate the warning LED.
- If the level reaches the critical region, activate the buzzer and critical LED.
- Send an alert message through the GSM module.
- Continue monitoring the water level.

Coding

```
#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

#include <SoftwareSerial.h>

// --- Wi-Fi Credentials ---

const char* ssid    = "Rithick";
const char* password = "12345678";

// --- ThingSpeak Configuration ---

WiFiClient client;

unsigned long myChannelNumber = 3459212;

const char* myWriteAPIKey  = "3PPRU6B6C7MBA334";
```



```
// --- GSM SIM800L Setup ---  
  
// RX Pin = D1 (GPIO5), TX Pin = D2 (GPIO4)  
  
SoftwareSerial gsmSerial(5, 4); // ESP RX, ESP TX  
  
const String PHONE_NUMBER = "+12345678900"; // Replace with recipient  
phone number with country code  
  
const float ALERT_DISTANCE_THRESHOLD = 10.0; // Distance threshold in  
cm to trigger SMS  
  
bool smsSent = false; // Flag to prevent sending repeated SMS continuously  
  
  
// --- Hardware Pin Definitions ---  
  
const int TRIG_PIN = 14; // D5 on NodeMCU / Wemos D1 Mini (GPIO14)  
const int ECHO_PIN = 12; // D6 on NodeMCU / Wemos D1 Mini (GPIO12)  
  
  
#define SOUND_SPEED 0.0343  
  
  
unsigned long lastThingSpeakUpdate = 0;  
const unsigned long thingspeakInterval = 15000; // 15 seconds  
  
  
void connectToWiFi() {  
    Serial.println();  
    Serial.print("Connecting to Wi-Fi network: ");  
    Serial.println(ssid);  
  
    WiFi.mode(WIFI_STA);  
    WiFi.begin(ssid, password);
```

```
while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
}
```

```
Serial.println("\n--- Wi-Fi Connected Successfully! ---");  
Serial.print("IP Address assigned: ");  
Serial.println(WiFi.localIP());  
Serial.print("Signal Strength (RSSI): ");  
Serial.print(WiFi.RSSI());  
Serial.println(" dBm\n");  
}
```

```
void initGSM() {  
    gsmSerial.begin(9600);  
    delay(1000);  
    Serial.println("Initializing GSM Module...");
```

```
// Handshake test
```

```
gsmSerial.println("AT");  
delay(1000);
```

```
// Set SMS text mode
```

```
gsmSerial.println("AT+CMGF=1");  
delay(1000);
```

```
Serial.println("GSM Module Ready.");  
}  
  
void sendSMS(String message) {  
    Serial.println("Sending SMS...");  
    gsmSerial.println("AT+CMGF=1"); // Ensure text mode  
    delay(500);  
  
    gsmSerial.print("AT+CMGS=\"");  
    gsmSerial.print(PHONE_NUMBER);  
    gsmSerial.println("\"");  
    delay(500);  
  
    gsmSerial.print(message);  
    delay(500);  
  
    gsmSerial.write(26); // ASCII character for Ctrl+Z to send SMS  
    delay(5000);  
  
    Serial.println("SMS sent successfully!");  
}  
  
float measureDistance() {  
    digitalWrite(TRIG_PIN, LOW);  
    delayMicroseconds(2);
```

```
digitalWrite(TRIG_PIN, HIGH);
```

```
delayMicroseconds(10);
```

```
digitalWrite(TRIG_PIN, LOW);
```

```
long duration = pulseIn(ECHO_PIN, HIGH, 30000);
```

```
if (duration == 0) {
```

```
    return -1.0;
```

```
}
```

```
return (duration * SOUND_SPEED) / 2.0;
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    delay(100);
```

```
    pinMode(TRIG_PIN, OUTPUT);
```

```
    pinMode(ECHO_PIN, INPUT);
```

```
// Initialize Wi-Fi & ThingSpeak
```

```
connectToWiFi();
```

```
ThingSpeak.begin(client);
```

```
// Initialize GSM Module
```

```
initGSM();
```

```
}
```

```
void loop() {
```

```
  if (WiFi.status() != WL_CONNECTED) {
```

```
    Serial.println("Wi-Fi connection lost. Reconnecting...");
```

```
    WiFi.reconnect();
```

```
    return;
```

```
  }
```

```
  float distanceCm = measureDistance();
```

```
  if (distanceCm < 0) {
```

```
    Serial.println("[Sensor] Target out of range.");
```

```
  } else {
```

```
    float distanceInch = distanceCm * 0.393701;
```

```
    Serial.printf("[Sensor] Distance: %.1f cm | %.1f in\n", distanceCm,  
distanceInch);
```

```
    // --- GSM SMS Logic ---
```

```
    if (distanceCm <= ALERT_DISTANCE_THRESHOLD && !smsSent) {
```

```
      String smsText = "ALERT: Object detected! Distance: " +  
String(distanceCm, 1) + " cm";
```

```
      sendSMS(smsText);
```

```
      smsSent = true; // Prevent spamming SMS repeatedly
```

```
    } else if (distanceCm > ALERT_DISTANCE_THRESHOLD) {
```

```
      smsSent = false; // Reset trigger when distance is back to normal
```

```
    }
```

```

    }

    // Upload to ThingSpeak every 15 seconds
    if (millis() - lastThingSpeakUpdate >= thingspeakInterval) {
        lastThingSpeakUpdate = millis();

        if (distanceCm >= 0) {
            ThingSpeak.setField(1, distanceCm);

            int httpCode = ThingSpeak.writeFields(myChannelNumber,
myWriteAPIKey);

            if (httpCode == 200) {
                Serial.println(">> ThingSpeak Update: Successful (HTTP 200)");
            } else {
                Serial.printf(">> ThingSpeak Update: Failed (HTTP Error Code %d)\n",
httpCode);
            }
        } else {
            Serial.println(">> ThingSpeak Update: Skipped (Out of range reading)");
        }
    }

    delay(1000);
}

```

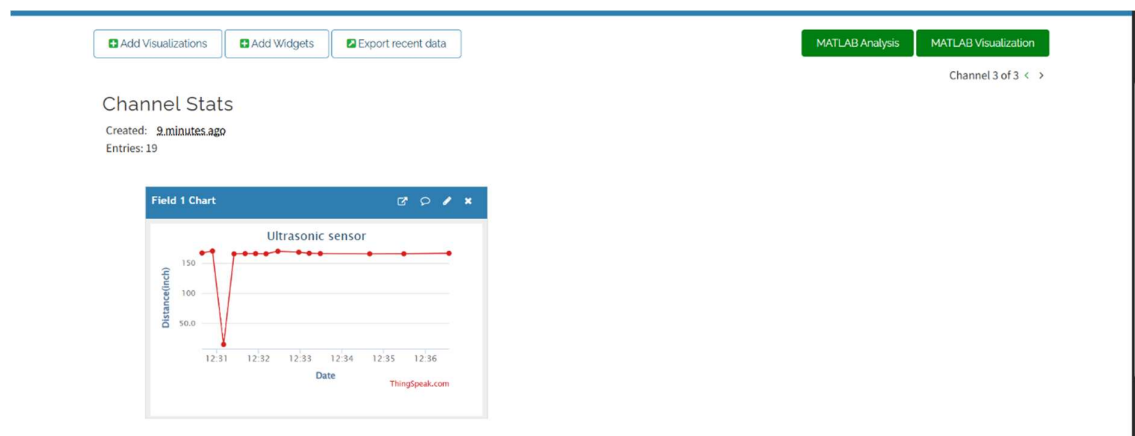
Coding output:

The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar contains icons for opening files, saving, compiling, uploading, and monitoring the serial port. The sketch is named "sketch_aug18b.ino" and is for a "Generic ESP8266 Module".

The code in the sketch is as follows:

```

//
// sensor test program
//
// 78
//    gsmSerial.print(message);
//    delay(500);
// 79
// 80    gsmSerial.write(26); // ASCII character for Ctrl+Z to send SMS
//    delay(5000);
// 81
// 82    Serial.println("SMS sent successfully!");
// 83
// 84    }
// 85
// 86    float measureDistance() {
// 87        // ultrasonic sensor module
// 88    }
// 89
// 90
// 91
// 92
// 93
// 94
// 95
// 96
// 97
// 98
// 99
// 100
// 101
// 102
// 103
// 104
// 105
// 106
// 107
// 108
// 109
// 110
// 111
// 112
// 113
// 114
// 115
// 116
// 117
// 118
// 119
// 120
// 121
// 122
// 123
// 124
// 125
// 126
// 127
// 128
// 129
// 130
// 131
// 132
// 133
// 134
// 135
// 136
// 137
// 138
// 139
// 140
// 141
// 142
// 143
// 144
// 145
// 146
// 147
// 148
// 149
// 150
// 151
// 152
// 153
// 154
// 155
// 156
// 157
// 158
// 159
// 160
// 161
// 162
// 163
// 164
// 165
// 166
// 167
// 168
// 169
// 170
// 171
// 172
// 173
// 174
// 175
// 176
// 177
// 178
// 179
// 180
// 181
// 182
// 183
// 184
// 185
// 186
// 187
// 188
// 189
// 190
// 191
// 192
// 193
// 194
// 195
// 196
// 197
// 198
// 199
// 200
// 201
// 202
// 203
// 204
// 205
// 206
// 207
// 208
// 209
// 210
// 211
// 212
// 213
// 214
// 215
// 216
// 217
// 218
// 219
// 220
// 221
// 222
// 223
// 224
// 225
// 226
// 227
// 228
// 229
// 230
// 231
// 232
// 233
// 234
// 235
// 236
// 237
// 238
// 239
// 240
// 241
// 242
// 243
// 244
// 245
// 246
// 247
// 248
// 249
// 250
// 251
// 252
// 253
// 254
// 255
// 256
// 257
// 258
// 259
// 260
// 261
// 262
// 263
// 264
// 265
// 266
// 267
// 268
// 269
// 270
// 271
// 272
// 273
// 274
// 275
// 276
// 277
// 278
// 279
// 280
// 281
// 282
// 283
// 284
// 285
// 286
// 287
// 288
// 289
// 290
// 291
// 292
// 293
// 294
// 295
// 296
// 297
// 298
// 299
// 300
// 301
// 302
// 303
// 304
// 305
// 306
// 307
// 308
// 309
// 310
// 311
// 312
// 313
// 314
// 315
// 316
// 317
// 318
// 319
// 320
// 321
// 322
// 323
// 324
// 325
// 326
// 327
// 328
// 329
// 330
// 331
// 332
// 333
// 334
// 335
// 336
// 337
// 338
// 339
// 340
// 341
// 342
// 343
// 344
// 345
// 346
// 347
// 348
// 349
// 350
// 351
// 352
// 353
// 354
// 355
// 356
// 357
// 358
// 359
// 360
// 361
// 362
// 363
// 364
// 365
// 366
// 367
// 368
// 369
// 370
// 371
// 372
// 373
// 374
// 375
// 376
// 377
// 378
// 379
// 380
// 381
// 382
// 383
// 384
// 385
// 386
// 387
// 388
// 389
// 390
// 391
// 392
// 393
// 394
// 395
// 396
// 397
// 398
// 399
// 400
// 401
// 402
// 403
// 404
// 405
// 406
// 407
// 408
// 409
// 410
// 411
// 412
// 413
// 414
// 415
// 416
// 417
// 418
// 419
// 420
// 421
// 422
// 423
// 424
// 425
// 426
// 427
// 428
// 429
// 430
// 431
// 432
// 433
// 434
// 435
// 436
// 437
// 438
// 439
// 440
// 441
// 442
// 443
// 444
// 445
// 446
// 447
// 448
// 449
// 450
// 451
// 452
// 453
// 454
// 455
// 456
// 457
// 458
// 459
// 460
// 461
// 462
// 463
// 464
// 465
// 466
// 467
// 468
// 469
// 470
// 471
// 472
// 473
// 474
// 475
// 476
// 477
// 478
// 479
// 480
// 481
// 482
// 483
// 484
// 485
// 486
// 487
// 488
// 489
// 490
// 491
// 492
// 493
// 494
// 495
// 496
// 497
// 498
// 499
// 500
// 501
// 502
// 503
// 504
// 505
// 506
// 507
// 508
// 509
// 510
// 511
// 512
// 513
// 514
// 515
// 516
// 517
// 518
// 519
// 520
// 521
// 522
// 523
// 524
// 525
// 526
// 527
// 528
// 529
// 530
// 531
// 532
// 533
// 534
// 535
// 536
// 537
// 538
// 539
// 540
// 541
// 542
// 543
// 544
// 545
// 546
// 547
// 548
// 549
// 550
// 551
// 552
// 553
// 554
// 555
// 556
// 557
// 558
// 559
// 560
// 561
// 562
// 563
// 564
// 565
// 566
// 567
// 568
// 569
// 570
// 571
// 572
// 573
// 574
// 575
// 576
// 577
// 578
// 579
// 580
// 581
// 582
// 583
// 584
// 585
// 586
// 587
// 588
// 589
// 590
// 591
// 592
// 593
// 594
// 595
// 596
// 597
// 598
// 599
// 600
// 601
// 602
// 603
// 604
// 605
// 606
// 607
// 608
// 609
// 610
// 611
// 612
// 613
// 614
// 615
// 616
// 617
// 618
// 619
// 620
// 621
// 622
// 623
// 624
// 625
// 626
// 627
// 628
// 629
// 630
// 631
// 632
// 633
// 634
// 635
// 636
// 637
// 638
// 639
// 640
// 641
// 642
// 643
// 644
// 645
// 646
// 647
// 648
// 649
// 650
// 651
// 652
// 653
// 654
// 655
// 656
// 657
// 658
// 659
// 660
// 661
// 662
// 663
// 664
// 665
// 666
// 667
// 668
// 669
// 670
// 671
// 672
// 673
// 674
// 675
// 676
// 677
// 678
// 679
// 680
// 681
// 682
// 683
// 684
// 685
// 686
// 687
// 688
// 689
// 690
// 691
// 692
// 693
// 694
// 695
// 696
// 697
// 698
// 699
// 700
// 701
// 702
// 703
// 704
// 705
// 706
// 707
// 708
// 709
// 710
// 711
// 712
// 713
// 714
// 715
// 716
// 717
// 718
// 719
// 720
// 721
// 722
// 723
// 724
// 725
// 726
// 727
// 728
// 729
// 730
// 
```



System Working

- The system is powered ON, and the **ESP8266 Node MCU** initializes the ultrasonic sensor, LEDs, buzzer, and GSM module.
- The **HC-SR04 ultrasonic sensor** continuously sends ultrasonic waves toward the water surface and receives the reflected echo.
- The ESP8266 calculates the **distance between the sensor and the water surface** using the echo time.

- As the water level rises, the distance measured by the ultrasonic sensor decreases.
- The ESP8266 compares the measured value with the predefined **safe, warning, and critical thresholds**.
- During the **normal water level**, the normal LED is activated and the buzzer remains OFF.
- When the water reaches the **warning level**, the warning LED is activated to indicate that the water level is increasing.
- When the water reaches the **critical level**, the warning LED and buzzer are activated to provide an immediate flood/overflow warning.
- At the critical level, the **GSM module sends an alert message** to the registered user.
- The system continuously repeats the measurement and alert process, allowing **real-time water-level monitoring and early flood warning**.

Bill of Materials

S. No.	Component	Specification	Quantity
1	ESP8266 Node MCU	Wi-Fi-enabled microcontroller	1
2	HC-SR04 Ultrasonic Sensor	Ultrasonic distance sensor	1
3	Buzzer	5V Active Buzzer	1
4	LED Indicators	Normal & Warning/Critical LEDs	2
5	GSM Module	GSM communication module	1
6	Breadboard	Standard solderless breadboard	1
7	Jumper Wires	Male-Male/Male-Female	1 Set
8	Resistors	Current-limiting resistors	As required
9	Voltage Divider	Resistors for level shifting	1 Set

Prototype Implementation

The prototype is implemented using an ESP8266 Node MCU as the main controller, with an HC-SR04 ultrasonic sensor mounted above a small water container to simulate a tank, river, or dam. The ultrasonic sensor continuously measures the distance between the sensor and the water surface and sends the readings to the ESP8266.

The two LED indicators are connected to the ESP8266 to display the water-level condition. The normal LED indicates a safe water level, while the warning LED indicates that the water level has reached a predefined warning or critical range. A buzzer is connected to provide an audible alert when the water reaches the critical level.

A GSM module is connected to the ESP8266 for remote notification. When the water level reaches the critical threshold, the ESP8266 activates the buzzer and sends an alert message through the GSM module to the registered mobile number.

Prototype Implementation Steps

- Place the ESP8266 Node MCU on the breadboard.
- Mount the HC-SR04 sensor securely above the water container.
- Connect the sensor's TRIG and ECHO pins to the assigned ESP8266 GPIO pins.
- Use a suitable voltage divider for the ECHO signal before connecting it to the ESP8266.
- Connect the normal and warning LEDs through suitable resistors.
- Connect the buzzer to the assigned GPIO pin.
- Connect the GSM module to the ESP8266 serial communication pins and provide its required power supply.
- Connect all required ground connections together.
- Upload the control program to the ESP8266.
- Set the safe, warning, and critical water-level thresholds in the program.
- Gradually add water to the container to test the different water-level conditions.

- Verify that the LEDs, buzzer, and GSM alert operate correctly when the water reaches the defined levels.

Hardware Setup:

S. No.	Hardware / Setup	Connection / Arrangement
1	ESP8266 Node MCU	Placed on breadboard
2	HC-SR04 Ultrasonic Sensor	Mounted above water container; TRIG → D5, ECHO → D6 through voltage divider
3	Normal LED	Connected to D1 through resistor
4	Warning LED	Connected to D2 through resistor
5	Buzzer	Connected to D7
6	GSM Module	TX/RX connected to ESP8266 serial pins
7	Breadboard	Components mounted and interconnected
8	Jumper Wires	Used between components and Node MCU
9	Water Container	HC-SR04 positioned above the container
10	Power Supply	Suitable supply provided to Node MCU and GSM module
11	Voltage Divider	Connected between HC-SR04 ECHO and ESP8266

Prototype Working

When the prototype is powered ON, the ESP8266 initializes all connected components. The ultrasonic sensor continuously measures the distance from the sensor to the water surface.

As the water level increases, the measured distance decreases. The ESP8266 compares the measured value with the programmed threshold levels. Under normal conditions, the normal LED remains active. When the water reaches the warning level, the warning LED is activated. At the critical level, the buzzer produces an audible warning and the GSM module sends an alert message to the predefined mobile number.

Expected Prototype Output:

Water Condition	Sensor Observation	LED Indication	Buzzer	GSM
Normal	Large distance from sensor to water	Normal LED ON	OFF	No alert
Warning	Water level rising	Warning LED ON	OFF/optional	Warning message
Critical	Very small distance	Critical LED ON	ON	Flood alert
Overflow Risk	Critical threshold exceeded	Critical LED ON	ON	Emergency alert

Prototype Demonstration

For demonstration, a transparent container can be used as a miniature water tank. The HC-SR04 sensor is fixed at the top of the container. Initially, the water level is kept low, so the system indicates the normal condition.

Water is then gradually added to the container. As the water rises, the distance measured by the ultrasonic sensor decreases. When the predefined warning level is reached, the warning LED turns ON. When the water reaches the critical level, the buzzer is activated and the GSM module sends an alert message. This demonstrates how the system can provide early warning before overflow.

Results & Performance Analysis

Parameter	Expected Performance
Water-level detection	Continuous
Monitoring method	Non-contact ultrasonic measurement
Local warning	LED + buzzer
Remote warning	GSM message
Controller	ESP8266 Node MCU
Response	Automatic
Monitoring	Repeated/continuous
Application	Tanks, dams, rivers, prototype flood monitoring

Expected Results:

- Continuous water-level monitoring.
- Accurate detection of changes in water level within the prototype setup.
- Automatic warning when the water level reaches the defined threshold.
- Audible alert during critical conditions.
- Visual indication through LEDs.
- GSM-based remote alert notification.
- Reduced dependency on manual monitoring.
- Early warning of potential overflow conditions.

Performance Analysis

Parameter	Performance Analysis
Water-Level Detection	The HC-SR04 continuously measures the distance between the sensor and water surface, allowing changes in water level to be detected.
Response Time	The ESP8266 processes sensor readings quickly and activates the warning devices when the predefined threshold is reached.
Alert System	The buzzer provides an immediate local warning during critical water levels.
LED Indication	Two LEDs provide a simple visual indication of safe and warning/critical conditions.
GSM Notification	The GSM module enables remote alert messages to be sent when a critical water level is detected.
Monitoring Capability	The system supports continuous monitoring without requiring manual observation.
Reliability	Performance is suitable for a prototype when the ultrasonic sensor is correctly positioned and protected from unsuitable environmental conditions.
Scalability	Different water-level thresholds can be programmed for tanks, reservoirs, and similar applications.
Overall Performance	The prototype provides an effective low-cost method for water-level monitoring and early overflow/flood warning.